

CONTINUATION-IN-PART: Lambda Recursion and Self-Invocation Architecture

Church-Turing Meta-Machine: O(1) Lambda Recursion Through Self-Invocation via CR6, Idempotent LAMBDA Re-Entry, and Multi-Paradigm Natural Language Compilation

Inventor: Kenneth James Hamer-Hodges

Filing Date: April 2026

Parent Application: Church-Turing Meta-Machine: Hardware-Enforced Lambda Calculus with the LAMBDA Instruction, NULL Capability Type, and Atomic Abstraction Architecture (Filed February 12, 2026)

Related Applications:

- Pure Church Lambda Processor: Architectural Exclusion of Turing-Domain Instructions as a Security Enforcement Mechanism (Filed February 2026)
- Church-Turing Meta-Machine: Dual-Gate Trusted Security Base with Hardware-Enforced Lambda Calculus, Deterministic Garbage Collection, and Architectural Vulnerability Elimination (Filed February 2026)
- Language-Independent Capability-Secured Instruction Set Architecture with Multi-Language Compiler (Filed March 2026)
- Abstract Golden Token I/O and Network Addressing Architecture (Filed March 2026)

Classification: Computer Architecture; Hardware Security; Capability-Based Computing; Lambda Calculus Processor; Recursion Optimization; Natural Language Compilation; Context-Switch Optimization

TITLE OF THE INVENTION

O(1) Lambda Recursion Through Idempotent Self-Invocation via Capability List Register in a Capability-Based Processor with Natural Language Compilation

CROSS-REFERENCE TO RELATED APPLICATIONS

This application is a continuation-in-part of the CTMM patent applications filed February–March 2026, which disclose: the Golden Token (GT) capability architecture with

domain purity enforcement; the LAMBDA instruction for lightweight in-scope code application with machine-status fast path; self-describing stack frames with 1-bit CALL/LAMBDA tag; non-nestable LAMBDA with CALL-mediated nesting; the CALL lump split creating CR14 (code, X-only) and CR6 (c-list, L-only); and the CLOOMC++ multi-language compiler with JavaScript and Haskell front-ends.

The present application extends those disclosures with five innovations:

1. Self-invocation via CR6 — A recursion mechanism where CALL CR6 and LAMBDA CR6 re-enter the current method using the capability list register that the CALL lump split already established, providing zero-cost self-reference without additional GT allocation or namespace lookup.
2. Idempotent LAMBDA re-entry — A refinement of the parent application's non-nestable LAMBDA rule, wherein LAMBDA CR6 is permitted to re-execute while `lambda_active` is already set, because the return address is invariant (the same value is overwritten with the same value). No hardware counter is needed — the software's own recursion argument (e.g., `n` counting down to 0) drives the recursion, and the existing 1-bit `lambda_active` flag plus `lambda_pc` register provide $O(1)$ exit: the base-case RETURN clears the flag and jumps to the instruction after LAMBDA CR6, where a second RETURN pops the CALL frame. Two RETURNS total, regardless of depth.
3. Three architectural loop styles — While loops (BRANCH), Recursive Repeat (CALL CR6), and Lambda Recursion (LAMBDA CR6) as distinct hardware mechanisms offering different security, performance, and resource tradeoff profiles, all compilable from the same source language.
4. Natural language compilation — An English front-end for the CLOOMC++ compiler that accepts plain English sentences ("Add a method called Sum that takes `n`", "While `n` is greater than 0", "Repeat with `n` minus 1") and compiles them to capability-secured Church Machine instructions.
5. Pet-name capability references — Named capability tokens (Pi, E, Phi, Zero, One) compiled as Abstract GT operations, enabling mathematical constant references in compiled code through the capability system.

FIELD OF THE INVENTION

The present invention relates to recursion optimization in a capability-based processor, wherein the architectural properties of the LAMBDA instruction — specifically, the invariance of the return address during self-invocation — enable idempotent re-entry without additional hardware state. The software's own recursion argument drives the countdown; the hardware's existing `lambda_active` flag and `lambda_pc` register handle the $O(1)$ exit path. No hardware counter, no stack frames, and no additional registers are required. The invention further relates to a self-invocation mechanism using the capability

list register (CR6) and to natural language compilation targeting the capability-secured instruction set.

BACKGROUND

The Recursion Cost Problem

All known processor architectures implement recursion through stack frames. Each recursive call pushes a return address (and typically saved registers) onto the stack. For N recursive calls, the architecture consumes $O(N)$ stack space, requires $O(N)$ time to unwind on return, and requires $O(N)$ time to save/restore on context switch. This cost applies regardless of whether the recursion is deep or shallow, and regardless of whether the return addresses are all the same or all different.

The Parent Architecture's Recursion Model

The parent CTMM application discloses two code invocation mechanisms:

- **CALL (E permission):** Crosses a protection domain boundary. Pushes a 2-word stack frame (E-GT + machine word with NIA). Performs namespace gate swap, creating new CR14 (code) and CR6 (c-list). Full mLoad validation on return. Heavyweight but secure.
- **LAMBDA (X permission):** Stays within the current protection domain. In the common case (no intervening CALL), uses a machine-status register pair (LAMBDA_PC and LAMBDA_active flag) with zero stack access. Lightweight but non-nestable — a second LAMBDA while LAMBDA_active is set causes a FAULT.

The parent application describes LAMBDA as non-nestable, with controlled nesting possible only through CALL mediation. Each nesting level requires a full CALL/RETURN pair. This is correct and secure, but imposes $O(N)$ overhead for recursive computations.

The Discovery: Return Address Invariance

The present invention recognizes that when a method invokes itself via CR6 using LAMBDA, every recursive call returns to the same address — the instruction immediately after the LAMBDA CR6 instruction. This invariance means that re-executing LAMBDA CR6 while lambda_active is already set is idempotent: it overwrites lambda_pc with the same value it already holds. The non-nestable FAULT is unnecessary for self-invocation because no new return context is created — the same return address is being written again.

The Key Insight: The Software Already Counts

The recursion depth is not hidden from the software — it is the software. The method's own argument (e.g., n in Sum(n)) counts down to zero. The software determines when to

stop (the base case). The hardware does not need to independently track depth because the software already does. The hardware only needs to know one thing: "am I in a LAMBDA body?" — which is the existing 1-bit `lambda_active` flag.

This insight eliminates the need for any hardware counter:

1. The software counter (`n`) drives the recursion
2. The base case (`n == 0`) triggers RETURN
3. RETURN sees `lambda_active = 1`, restores PC from `lambda_pc`, clears the flag
4. The restored PC points to the instruction after LAMBDA CR6, which is RETURN
5. This second RETURN sees `lambda_active = 0`, pops the CALL frame
6. Two RETURNS. Always two. Regardless of recursion depth.

The Self-Invocation Discovery

The parent application discloses the CALL lump split: when CALL processes an E-GT, it creates CR14 (code region, X-only) and CR6 (c-list region, L-only). The present invention recognizes that CR6 — which CALL has already established to point at the current method's c-list — can serve as a self-reference for recursion.

- CALL CR6: Invokes the current method again through the full CALL path (namespace gate swap, 2-word frame, capability re-validation). This is Recursive Repeat — secure, predictable, heavyweight.
- LAMBDA CR6: Invokes the current method again through the lightweight LAMBDA path (no namespace swap, machine-status register, X permission check only). This is Lambda Recursion — the lightest possible self-invocation.

In both cases, CR6 was already set up by the initial CALL that entered the method. No additional GT allocation, no namespace lookup, no c-list modification is required. The self-reference is a free consequence of the architecture.

DETAILED DESCRIPTION

1. Self-Invocation via CR6

1.1 The Mechanism

When a method is entered via CALL, the CALL lump split creates:

- CR14: Code region (X-only), pointing to the method's instruction space
- CR6: C-list region (L-only), pointing to the method's capability list

CR6 holds an Inform GT with L (Load) permission pointing to the method's c-list base. The c-list IS the method's entry point for self-reference — because CALL CR6 will re-enter the same lump, re-split it, and re-create the same CR14 and CR6.

CALL CR6 (Recursive Repeat):

```
Step 1: Read CR6 – Inform GT with L permission, pointing to current method's c-list
Step 2: CALL processes CR6 as an E-GT (the method's lump has E permission)
Step 3: CALL performs lump split: creates new CR14 and CR6 (identical to current)
Step 4: Push 2-word frame (E-GT + machine word with NIA)
Step 5: Branch to method entry (offset 0 in new CR14)
Step 6: Method body executes with fresh CR14/CR6 (same values, new frame)
Step 7: RETURN pops frame, restores caller's CR14/CR6, validates via mLoad
```

LAMBDA CR6 (Lambda Recursion — Idempotent Re-Entry):

```
Step 1: Read CR6 – Inform GT with X permission (code body in same domain)
Step 2: LAMBDA verifies X permission on CR6
Step 3: If lambda_active == 0:
    Save return address (PC+4) to lambda_pc
    Set lambda_active = 1
    If lambda_active == 1:
        Overwrite lambda_pc with PC+4 (same value – idempotent)
Step 4: Branch to code entry point referenced by CR6
Step 5: Method body executes (same CR14, same CR6 – no namespace swap)
Step 6: On base case RETURN:
    lambda_active = 1 → PC ← lambda_pc, clear lambda_active
    Falls to instruction after LAMBDA CR6
Step 7: Second RETURN:
    lambda_active = 0 → real RETURN, pop CALL frame
```

1.2 The Idempotent Re-Entry Rule

The parent application's non-nestable rule states: if `lambda_active` is set and a LAMBDA instruction executes, the hardware generates a FAULT. The present invention refines this rule with a self-invocation exception:

Refined Rule: If `lambda_active` is set and LAMBDA CR6 executes (self-invocation), the hardware permits re-entry because the return address is invariant — `lambda_pc` is overwritten with the same value it already holds. This is not nesting (no new return context is created); it is re-entry to the same body with updated data register arguments.

The FAULT is preserved for LAMBDA to a different target (different CR, different return address) while `lambda_active` is set. The distinction is:

- LAMBDA CR6 while active → permit (idempotent, same return address)
- LAMBDA CR_n ($n \neq 6$) while active → FAULT (different return address, true nesting)

This requires no new hardware — it is a refinement of the existing FAULT condition logic, gating on whether the target CR is CR6.

1.3 Traced Execution: LambdaSum(3, 0)

```
CALL enters LambdaSum → push CALL frame, lump split creates CR14 + CR6
    lambda_active = 0
```

```
Level 0: DR_n=3, DR_total=0
    n≠0, skip base case
    total = 0+3 = 3, n = 3-1 = 2
    LAMBDA CR6: lambda_active was 0 → set lambda_active=1, lambda_pc=addr_after_lambda
    Branch to method entry
```

```
Level 1: DR_n=2, DR_total=3
```

```

n≠0, skip base case
total = 3+2 = 5, n = 2-1 = 1
LAMBDA CR6: lambda_active already 1 → overwrite lambda_pc with same value (idempotent)
Branch to method entry

Level 2: DR_n=1, DR_total=5
n≠0, skip base case
total = 5+1 = 6, n = 1-1 = 0
LAMBDA CR6: lambda_active already 1 → idempotent
Branch to method entry

Level 3: DR_n=0, DR_total=6
n==0! Base case.
RETURN #1: lambda_active=1 → PC ← lambda_pc, clear lambda_active
DR_total=6 (correct answer!)
Now at addr_after_lambda

RETURN #2: lambda_active=0 → real RETURN
Pop CALL frame from initial entry
Back to caller with DR_total=6 ✓

Total: 2 RETURNS for depth 3. Same 2 RETURNS for depth 3,000,000.
No counter. No stack frames. No unwinding.

```

1.4 Why No Hardware Counter Is Needed

The initial design proposed a 16-bit hardware counter (`lambda_depth_reg`) to track recursion depth. Analysis reveals this counter is unnecessary:

1. The software already counts: The method's own argument (`n`) is the recursion counter. It counts down to zero. The hardware does not need to independently replicate this count.
2. The counter would be discarded: When the base-case RETURN fires, the proposed counter would be zeroed and thrown away. It served no purpose during execution — the software determined when to stop.
3. The existing flag is sufficient: The 1-bit `lambda_active` flag already tells RETURN whether to take the LAMBDA fast path. No additional depth information is needed because the fast path behavior is the same regardless of depth: restore PC from `lambda_pc`, clear flag, done.
4. Two RETURNS is already $O(1)$: The base-case RETURN clears the flag ($O(1)$), the follow-on RETURN pops the CALL frame ($O(1)$). Total: $O(1)$ regardless of depth. A counter cannot improve on this.
5. Context switch is already $O(1)$: CHANGE saves `lambda_active` (1 bit) and `lambda_pc` (already packed in the NIA word) — two fixed-size values regardless of recursion depth. No counter needed.

The simpler design — just relaxing the FAULT rule for CR6 self-invocation — achieves the same $O(1)$ trifecta with zero additional hardware.

1.5 Compiler Primitives

The CLOOMC++ compiler provides two primitives for self-invocation:

- `recall(args)`: Emits LOAD instructions for updated arguments followed by CALL CR6. English syntax: "Repeat with x, y", "Recurse with n minus 1", "Call self with a, b", "Call again with n, total".
- `relambda(args)`: Emits LOAD instructions for updated arguments followed by LAMBDA CR6. English syntax: "Apply lambda with x, y", "Lambda repeat with n minus 1", "Lambda recurse with a, b", "Lambda self with count".

1.6 Security Properties — CALL CR6 Adds No Security Over LAMBDA CR6

A critical architectural observation: CALL CR6 is no more secure than LAMBDA CR6 for self-invocation. Both re-enter the same method, in the same protection domain, with the same c-list, accessing the same capabilities.

- CALL CR6 performs full namespace gate re-validation on each recursive call — but the gate re-validates the same GT, re-splits the same lump, creates the same CR14 and CR6, and enters the same code body. The re-validation is redundant work that produces an identical result every time. No new capabilities are introduced. No new domain is entered. The "security" of per-iteration re-validation is illusory when the target never changes.
- LAMBDA CR6 verifies X permission before branching — once. Since every re-entry targets the same code body with the same capabilities in the same domain, the X check on the first entry is sufficient. Subsequent idempotent re-entries execute the same verified code.
- No GT forgery in either case: CR6 was established by the original CALL lump split. Software cannot modify CR6's GT — only CALL and mLoad can write to capability registers. Both CALL CR6 and LAMBDA CR6 use a GT that was already validated at method entry.
- Idempotent re-entry is safe: Each re-entry executes the same code body with the same capabilities. Only data registers change (the arguments). The capability register file is untouched by LAMBDA execution — this is the Golden Rule strengthened, as disclosed in the parent application.

Conclusion: CALL CR6 recursion is strictly inferior to LAMBDA CR6 recursion — identical security, but $O(N)$ stack cost, $O(N)$ context-switch cost, and $O(N)$ unwind cost versus $O(1)$ for all three with LAMBDA CR6. CALL CR6 is never the preferred choice for self-invocation. It exists in the architecture because CALL can target any CR (including CR6), but when the intent is recursion, LAMBDA CR6 dominates on every axis. The only reason to describe CALL CR6 recursion is to demonstrate the superiority of LAMBDA CR6 by contrast.

2. The $O(1)$ Trifecta — Without a Counter

2.1 $O(1)$ Entry

Each LAMBDA CR6 re-entry requires:

- X permission check on CR6 (combinational — no memory access)
- Overwrite `lambda_pc` with `PC+4` (same value — idempotent write to existing register)
- Branch to method entry

No stack frame push. No counter increment. No memory access. One clock cycle beyond the normal LAMBDA path.

2.2 O(1) Context Switch

On CHANGE (thread context switch), the hardware saves:

- `lambda_active` (1 bit) — already packed in the NIA word's indicator bits
- `lambda_pc` (32 bits) — already saved as part of machine status

These are two fixed-size values regardless of recursion depth. Whether the method has recursed 1 time or 1,000,000 times, CHANGE saves the same two values. Compare with CALL CR6 recursive repeat, where each of N CALL frames must be individually saved.

2.3 O(1) Exit

The base case triggers two RETURNS:

RETURN #1 (from base case):

- `lambda_active = 1` → restore PC from `lambda_pc`, clear `lambda_active`
- This is the existing LAMBDA fast path from the parent application
- One cycle, no stack access

RETURN #2 (from after-LAMBDA instruction):

- `lambda_active = 0` → real RETURN, pop CALL frame
- This is the normal CALL RETURN path
- Standard frame pop

Total: exactly 2 RETURN instructions regardless of recursion depth.

For recursion depth N, this eliminates N-1 RETURN instructions compared to an architecture that unwinds iteratively. For N = 1,000,000, this saves 999,999 RETURN cycles.

2.4 The Counter as Optional Enhancement

While the core mechanism requires no hardware counter, an optional counter register (`lambda_depth_reg`, 16 bits) could serve diagnostic purposes:

- Performance monitoring: Query recursion depth without walking the stack
- Safety limit: Generate a FAULT on counter overflow (e.g., depth > 65535) to catch runaway recursion before stack exhaustion on the CALL side
- Debugging: Hardware watchpoint on counter value

These are enhancement features, not requirements for correctness or O(1) performance.

The base mechanism — idempotent LAMBDA re-entry with the existing flag and PC register — is complete without the counter.

3. Three Architectural Loop Styles

3.1 The Discovery

The combination of BRANCH, CALL CR6, and LAMBDA CR6 provides three distinct loop mechanisms with different security, performance, and resource profiles — all compiling from the same source code, all producing correct results, but with fundamentally different hardware behavior.

3.2 Comparison

Property	While (BRANCH)	CALL CR6 (shown for contrast)	LAMBDA CR6 (optimal)
Opcode	MCMP + BRANCH	CALL	LAMBDA
Loop mechanism	Compare-and-branch	Self-invocation with full frame	Idempotent self-invocation
Stack per iteration	0	2 words (SZ=1)	0 (no stack, no counter)
Namespace gate swap	No	Yes (redundant — same GT every time)	None
Branch prediction	Required (misprediction risk)	Not required (target is CR6, known)	Not required (target is CR6, known)
Speculative execution	Yes (Spectre/Meltdown risk)	No	No
Context switch cost	O(1)	O(N) (save N frames)	O(1) (save flag + PC)
Unwind cost	O(1)	O(N) (pop N frames)	O(1) (2 RETURNS always)
Pipeline stall risk	Yes (misprediction on final iteration)	No	No
Security vs LAMBDA	Requires branch analysis	No additional security (same method, same GT, same CR6)	None (same method, same GT, same CR6)
Instructions per iteration	4+ (MCMP, BRANCH, body, BRANCH)	2+ (args, CALL)	2+ (args, LAMBDA)
Additional hardware	Branch predictor	Stack memory	None (existing flag + PC)
Verdict	Vulnerable (Spectre/Meltdown)	Strictly inferior to LAMBDA CR6	Optimal for recursion

3.3 Architectural Significance — LAMBDA CR6 Dominates

The three loop styles exist in the architecture, but they are not equal alternatives — they form a hierarchy where LAMBDA CR6 is the clear winner for recursive self-invocation:

- While (BRANCH): Familiar to imperative programmers. Most compact loop code. But inherits all of conventional computing's branch prediction problems: pipeline stalls on misprediction, speculative execution vulnerabilities (Spectre, Meltdown), non-deterministic timing. These are the exact vulnerabilities the Church Machine is designed to eliminate.
- CALL CR6 (Recursive Repeat): Performs full namespace gate re-validation on every iteration — but this re-validation is redundant because CR6 always points to the same method. The gate re-checks the same GT, re-splits the same lump, and produces the same CR14 and CR6 every time. CALL CR6 adds no security over LAMBDA CR6 while imposing O(N) stack, O(N) context switch, and O(N) unwind costs. CALL CR6 is never the preferred choice for recursion. It is included in this comparison solely to demonstrate, by contrast, the power of LAMBDA CR6.

- LAMBDA CR6 (Lambda Recursion): $O(1)$ everything — entry, context switch, exit. No branch prediction, no speculative execution, no namespace swap. No additional hardware beyond the existing `lambda_active` flag and `lambda_pc` register. Same security as CALL CR6 (same method, same domain, same capabilities) with none of the overhead. The optimal recursion primitive.

The architectural lesson: CALL is the correct instruction for cross-domain invocation (entering a different abstraction with a different c-list). When the target is the same method (CR6), CALL's heavyweight machinery provides no benefit — it is LAMBDA's domain. The CALL lump split creates CR6 as a self-reference; LAMBDA CR6 is how that self-reference should be used.

4. Natural Language Compilation (English Front-End)

4.1 The Extension

The parent CLOOMC++ patent application discloses JavaScript (imperative) and Haskell (functional) front-ends compiling to the 20-instruction Church Machine ISA. The present invention adds an English front-end — the third paradigm — that accepts plain English sentences and compiles them to the same capability-secured instruction set.

4.2 English Syntax

The English front-end parses structured English sentences with the following mappings:

English Source	Church Machine Instruction(s)
Add a method called X that takes n	Method prologue (register allocation for parameters)
Set total to 0	IADD DRx, DR0, #0 (DR0 = hardwired zero)
Set total to total plus n	IADD DRx, DRy, DRz
Set n to n minus 1	ISUB DRx, DRy, #1
While n is greater than 0	MCMP DRx, #0 + BRANCH.LE (skip to End while)
End while	BRANCH (jump back to While)
If n is equal to 0	MCMP DRx, #0 + BRANCH.NE (skip to End if)
End if	(branch target)
Return total	Move result to DR1 + RETURN AL
Repeat with n, total	LOAD args + CALL CR6 (recall)
Apply lambda with n, total	LOAD args + LAMBDA CR6 (relambda)

4.3 Condition Phrases

The English compiler recognizes six comparison phrases, each mapping to an MCMP condition code:

English Phrase	MCMP Condition
is greater than	GT
is less than	LT
is equal to / equals	EQ
is not equal to	NE

is greater than or equal to	GE
is less than or equal to	LE

4.4 Universal Target Proof

The English front-end produces the same instructions as the JavaScript and Haskell front-ends for equivalent computations:

- English: Set total to total plus n $\rightarrow$ IADD DRx, DRy, DRz
- JavaScript: total = total + n $\rightarrow$ IADD DRx, DRy, DRz
- Haskell: total + n $\rightarrow$ IADD DRx, DRy, DRz

The hardware cannot distinguish which language produced the code. This extends the universal computation target property to three paradigms: imperative (JavaScript), functional (Haskell), and natural language (English).

4.5 Significance for the Three Loop Styles

The English front-end is the first natural language compiler to directly expose all three architectural loop styles:

- While: While n is greater than 0 ... End while $\rightarrow$ MCMP + BRANCH
- Recursive Repeat: Repeat with n, total $\rightarrow$ CALL CR6
- Lambda Recursion: Apply lambda with n, total $\rightarrow$ LAMBDA CR6

A programmer writing in plain English can choose between conventional looping, secure recursion, and lightweight lambda recursion — without knowing assembly language, register names, or opcode encodings.

5. Pet-Name Capability References

5.1 The Mechanism

The CLOOMC++ compiler recognizes mathematical constant names (Pi, E, Phi, Zero, One) and compiles them as references to Abstract Golden Tokens in the Lambda abstraction's c-list. Each constant is an Abstract GT (gt_type = 11₂) whose word1_location encodes the constant's hardware identity.

5.2 Compilation

```
Source: Load Pi
Target: LOAD CR_temp, CR6, #offset_of_Pi_in_clist
        ; CR_temp now holds Abstract GT for Pi
        ; Value encoded in word1_location, immutable, unforgeable
```

5.3 Significance

Pet-name capability references demonstrate that the Abstract GT type — originally designed for I/O addressing — also serves as a mechanism for mathematical constants. The constant's value is architecturally immutable (it is the GT, not a memory location),

unforgeable (no instruction can synthesize a GT), and capability-secured (requires L permission on CR6 to access).

REDUCTION TO PRACTICE

Self-Invocation Proof

The CLOOMC++ compiler (simulator/cloomc_compiler.js) implements recall() and relambda() primitives. The following three methods compute the sum $1+2+\dots+n$ using all three loop styles:

WhileSum (BRANCH, 12 instructions):

```
Add a method called WhileSum that takes n
Set total to 0
While n is greater than 0
    Set total to total plus n
    Set n to n minus 1
End while
Return total
```

Compiles to: IADD, MCMP, BRANCH.LE, IADD, ISUB, BRANCH, RETURN — 2 branches per iteration.

RecurseSum (CALL CR6, 10 instructions):

```
Add a method called RecurseSum that takes n and total
If n is equal to 0
    Return total
End if
Set total to total plus n
Set n to n minus 1
Repeat with n, total
```

Compiles to: MCMP, BRANCH.NE, RETURN, IADD, ISUB, LOAD, LOAD, CALL CR6 — 1 CALL per iteration, 0 branches for looping.

LambdaSum (LAMBDA CR6, 10 instructions):

```
Add a method called LambdaSum that takes n and total
If n is equal to 0
    Return total
End if
Set total to total plus n
Set n to n minus 1
Apply lambda with n, total
```

Compiles to: MCMP, BRANCH.NE, RETURN, IADD, ISUB, LOAD, LOAD, LAMBDA CR6 — 1 LAMBDA per iteration, 0 branches for looping, zero stack overhead (idempotent re-entry, no frames pushed).

All three methods produce $\text{Sum}(5) = 15$. The compiled code has been verified in the Church Machine simulator.

Idempotent Re-Entry Verification

The traced execution of LambdaSum(3, 0) demonstrates:

- 3 levels of LAMBDA CR6 re-entry with lambda_active already set
- Each re-entry overwrites lambda_pc with the same value (idempotent)
- Base case triggers 2 RETURNS: flag-clear + CALL-frame-pop
- Result: $DR_total = 6 = 1+2+3$
- Total RETURNS: 2 (same for depth 3, 30, 300, or 3,000,000)

English Front-End Proof

The English front-end has compiled all three loop methods above, plus additional methods, to correct Church Machine instruction sequences. Language detection identifies English by the presence of "Add a method", "Set x to", "While", "Repeat with", "Apply lambda with", and other English markers.

PROPOSED CLAIMS

Claim 1 — Self-Invocation via Capability List Register (Independent)

A recursion mechanism in a capability-based processor comprising:

- (a) a CALL instruction that, upon entering an abstraction, creates a code region capability register (CR14) and a capability list register (CR6) by splitting the abstraction's memory lump;
- (b) wherein the capability list register (CR6) holds an Inform Golden Token pointing to the abstraction's own entry point, providing a self-reference as a free consequence of the CALL lump split — no additional GT allocation, namespace lookup, or c-list modification is required;
- (c) wherein a LAMBDA instruction targeting CR6 re-enters the same abstraction with X permission verification only, no namespace gate swap, and no stack frame push — implementing optimal recursive self-invocation with $O(1)$ entry, $O(1)$ context switch, and $O(1)$ exit;
- (d) wherein a CALL instruction targeting CR6 also re-enters the same abstraction but with full namespace gate re-validation, a new 2-word stack frame, and $O(N)$ cost — providing no additional security over LAMBDA CR6 because the re-validation re-checks the same GT, re-splits the same lump, and enters the same domain every time;
- (e) wherein the architectural analysis demonstrates that LAMBDA CR6 dominates CALL CR6 for self-invocation on every axis: identical security (same method, same domain, same capabilities), but $O(1)$ cost versus $O(N)$ — establishing LAMBDA CR6 as the optimal and preferred recursion primitive.

Claim 2 — Idempotent LAMBDA Re-Entry for O(1) Recursion (Independent)

A hardware mechanism for recursive self-invocation in a capability-based processor, comprising:

- (a) a LAMBDA instruction that, when targeting the capability list register (CR6) while the `lambda_active` flag is already set, is permitted to re-execute without generating a FAULT — because the return address written to `lambda_pc` is invariant (always PC+4 of the same LAMBDA CR6 instruction), making the re-entry idempotent;
- (b) wherein the method's own recursion argument (e.g., a counter `n` counting down to zero in data registers) drives the recursion — the hardware does not independently track recursion depth;
- (c) wherein the base-case RETURN instruction, upon detecting `lambda_active = 1`, restores PC from `lambda_pc` (the instruction after LAMBDA CR6) and clears `lambda_active` — in a single cycle with no stack access;
- (d) wherein a second RETURN instruction, now finding `lambda_active = 0`, performs the real RETURN by popping the CALL frame from the initial method entry;
- (e) wherein exactly two RETURN instructions execute regardless of recursion depth — achieving O(1) exit for arbitrarily deep recursion with no hardware counter, no stack unwinding, and no iterative frame popping;
- (f) wherein the FAULT is preserved for LAMBDA to a different target (different CR, different return address) while `lambda_active` is set — the idempotent exception applies exclusively to CR6 self-invocation;
- (g) thereby achieving O(1) recursion entry (one idempotent register write), O(1) context switch (save `lambda_active` flag + `lambda_pc` — two fixed-size values regardless of depth), and O(1) exit (two RETURNS always) — using only the existing `lambda_active` flag and `lambda_pc` register with no additional hardware.

Claim 3 — Two-RETURN Exit Path (Dependent on Claim 2)

The processor of Claim 2, wherein:

- (a) the code structure of a LAMBDA CR6 recursive method places the LAMBDA CR6 instruction as the final executable statement before the method's trailing RETURN instruction;
- (b) the base-case RETURN (#1) clears `lambda_active` and jumps to the instruction after LAMBDA CR6, which is the method's trailing RETURN;
- (c) the trailing RETURN (#2) finds `lambda_active = 0` and pops the CALL frame from the initial method entry;
- (d) thereby, the two-RETURN exit path is a structural consequence of the method layout

— the first RETURN navigates to the second RETURN via `lambda_pc`, and the second RETURN exits the method via the CALL frame — regardless of whether recursion depth was 1, 100, or 1,000,000.

Claim 4 — Three Architectural Loop Styles Demonstrating LAMBDA CR6 Dominance (Independent)

A processor architecture providing three distinct loop mechanisms within a single instruction set, wherein analysis demonstrates that LAMBDA CR6 is the optimal recursion primitive:

(a) a compare-and-branch loop (While) using MCMP and BRANCH instructions, which is familiar to imperative programming but requires branch prediction, is susceptible to pipeline stalls on misprediction, and enables speculative execution vulnerabilities (Spectre, Meltdown) — the exact class of vulnerabilities the Church Machine eliminates;

(b) a recursive repeat (CALL CR6) which invokes the current method through the full CALL path with namespace gate re-validation on every iteration — but wherein the re-validation is redundant because CR6 always points to the same method, re-checking the same GT and re-splitting the same lump, adding $O(N)$ stack and context-switch cost with no security benefit over LAMBDA CR6;

(c) a lambda recursion (LAMBDA CR6) which invokes the current method through the idempotent LAMBDA re-entry path of Claim 2, with X permission check only, no branch prediction, no speculative execution, no namespace swap, no stack frames, and $O(1)$ entry, context switch, and exit — requiring no hardware beyond the existing `lambda_active` flag and `lambda_pc` register;

(d) wherein all three mechanisms compile from the same source language to the same instruction set and produce the same computational result, but LAMBDA CR6 dominates: it provides the same security as CALL CR6 (same method, same domain, same capabilities) at $O(1)$ cost instead of $O(N)$, while eliminating all branch-prediction vulnerabilities that afflict While loops;

(e) wherein LAMBDA CR6 requires zero additional hardware — the existing `lambda_active` flag and `lambda_pc` register, already present for single-level LAMBDA in the parent application, are the complete recursion mechanism. CALL CR6 recursion exists only as an architectural consequence (CALL can target any CR) and serves as a foil demonstrating the power of LAMBDA CR6.

Claim 5 — Natural Language Compilation to Capability-Secured Instructions (Independent)

A compilation method for a capability-secured processor, comprising:

(a) an English-language front-end that parses structured English sentences including method declarations ("Add a method called X that takes n"), variable assignments ("Set total to total plus n"), conditional blocks ("While n is greater than 0 ... End while", "If n is

equal to 0 ... End if"), and recursive self-invocation ("Repeat with n, total", "Apply lambda with n, total");

(b) wherein the English front-end compiles to the same capability-secured instruction set as the JavaScript (imperative) and Haskell (functional) front-ends disclosed in the parent applications;

(c) wherein the English front-end exposes all three loop styles of Claim 4 through natural language syntax: "While ... End while" for compare-and-branch, "Repeat with" for secure recursive repeat, and "Apply lambda with" for lightweight lambda recursion;

(d) wherein the compiled output is indistinguishable from the output of the JavaScript or Haskell front-ends for equivalent computations — the hardware cannot determine the source language;

(e) thereby extending the universal computation target property of the Church Machine instruction set to three programming paradigms: imperative, functional, and natural language.

Claim 6 — Invariant Return Address as Idempotent Re-Entry Precondition (Dependent on Claim 2)

The processor of Claim 2, wherein the correctness of idempotent re-entry depends on a structural invariant:

(a) every LAMBDA CR6 instruction within a given method body branches to the same target (the method's own code entry point, as established by CR6);

(b) every LAMBDA CR6 instruction writes the same return address to `lambda_pc` (the instruction immediately following the LAMBDA CR6 instruction);

(c) the return address is invariant across all recursion levels — it does not change with recursion depth — making each re-entry write idempotent;

(d) therefore re-entry does not create new return context — the `lambda_pc` register holds the same value before and after the write — and no stack, counter, or additional state is needed;

(e) this invariant holds exclusively for LAMBDA CR6 self-invocation; LAMBDA to a different target writes a different return address and would corrupt the existing `lambda_pc`, which is why the non-nestable FAULT is preserved for non-CR6 targets.

Claim 7 — Pet-Name Mathematical Constants via Abstract GT (Dependent on Parent I/O Patent)

A method of representing mathematical constants in a capability-based processor, comprising:

(a) each constant (Pi, E, Phi, Zero, One) is represented as an Abstract Golden Token

(gt_type = 11₂) in the abstraction's c-list;

(b) the compiler resolves the constant name to its c-list offset and emits a LOAD instruction to retrieve the Abstract GT into a capability register;

(c) the constant's value is encoded in the Abstract GT's word1_location field and is architecturally immutable — no instruction can modify it;

(d) the constant is unforgeable — no instruction can synthesize a GT, and the constant can only be accessed through capability-mediated LOAD with L permission on CR6;

(e) thereby providing mathematical constants as first-class capability-secured values, accessed through the same GT mechanism used for I/O peripherals and network tunnels.

PRIOR ART DISTINCTION

Recursion in Prior Architectures

System	Self-Invocation	Idempotent Re-Entry	Entry Context Switch	One-Exit	Three Loop Styles	Natural Language	Additional Hardware
x86/ARM	Computed branch	No	No	No	No	No	Branch predictor
MIPS	JAL/JR	No	No	No	No	No	Return address reg
LISP Machines	TCO (compiler)	No	No	No	No	No	Stack
Reduceron	Graph reduction	No	No	No	No	No	Heap
CHERI	Same as base ISA	No	No	No	No	No	Capability cache
CTMM (parent)	CALL only (LAMBDA FAULTS)	No	No	No	No	No	—
CTMM (this CIP)	CALL CR6 + LAMBDA CR6	Yes	Yes	Yes (2 RETURNS)	Yes	Yes	None

Tail-Call Optimization (TCO) — Closest Prior Art

Functional language implementations (Scheme, Haskell, ML) perform tail-call optimization (TCO), which reuses the current stack frame instead of allocating a new one. TCO achieves O(1) stack space for tail-recursive functions.

Distinction: TCO is a compiler optimization that modifies generated code. The present invention is a hardware mechanism that:

1. Does not require compiler participation — the idempotent re-entry operates at the hardware level by relaxing the FAULT condition for CR6
2. Provides O(1) context switch — TCO does not address context switch cost
3. Provides O(1) exit via the two-RETURN path — TCO eliminates the return entirely (tail position), but the present invention preserves the RETURN instruction semantics while making exit cost depth-independent
4. Works within a capability-secured instruction set — TCO operates in unprotected address spaces
5. Requires no additional hardware — the existing lambda_active flag and lambda_pc

register, already present for single-level LAMBDA, are the complete mechanism. TCO typically requires compiler analysis and code transformation.

The idempotent re-entry mechanism is architecturally invisible to the instruction stream — the same LAMBDA CR6 instruction executes on each recursion level. The O(1) behavior is a consequence of the return address invariance, not a compiler transformation.

FIGURES (Proposed)

Figure 1: LAMBDA CR6 Idempotent Self-Invocation Flow

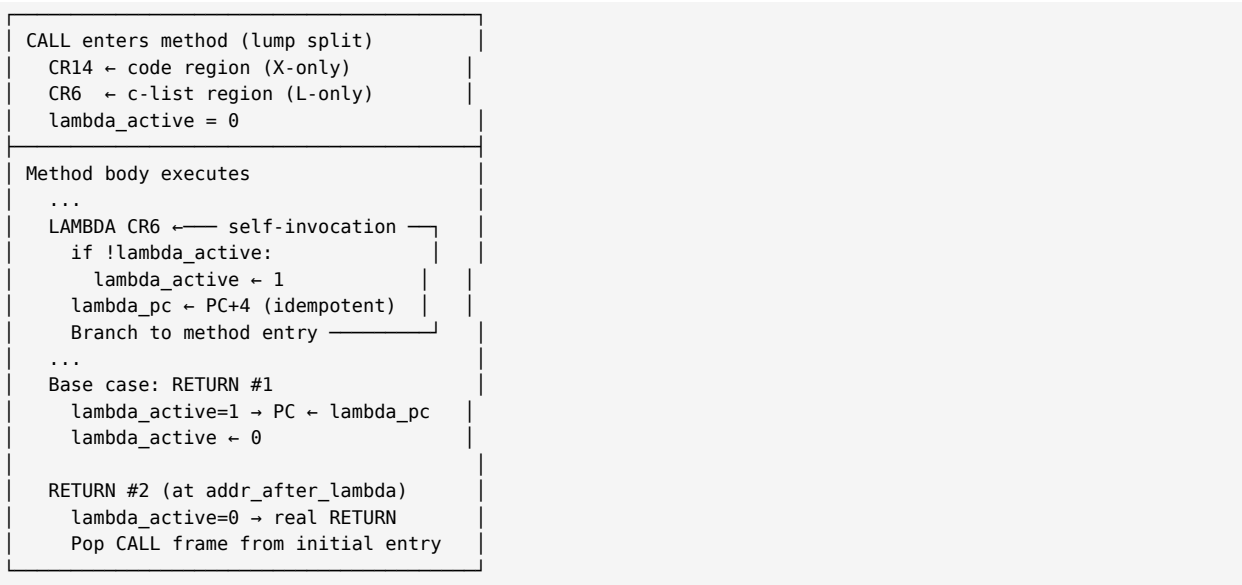
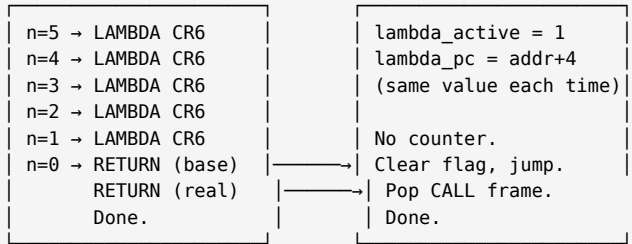


Figure 2: Three Loop Styles — LAMBDA CR6 Dominates

While (BRANCH)	CALL CR6 (for contrast)	LAMBDA CR6 (OPTIMAL)
MCMP + BRANCH ← body MCMP + BRANCH	CALL CR6 body CALL CR6 ... RETURN RETURN RETURN	LAMBDA CR6 body LAMBDA CR6 ... RETURN #1 (base) clear flag RETURN #2 (real)
Stack: 0	Stack: 2N words	Stack: 0
Branches: 2/iter	Branches: 0	Branches: 0
Pipeline: stall risk	Pipeline: deterministic	Pipeline: deterministic
Unwind: O(1)	Unwind: O(N)	Unwind: O(1) – always 2
Change: O(1)	Change: O(N)	Change: O(1)
New HW: predictor	New HW: stack memory	New HW: NONE
Security: vulnerable	Security: SAME AS LAMBDA	Security: X perm at entry
Verdict: Spectre risk	Verdict: STRICTLY INFERIOR	Verdict: OPTIMAL

Figure 3: Why No Counter — Software Drives, Hardware Follows

Software (data registers): Hardware (machine status):



Software counts.

Hardware doesn't need to.

ABSTRACT

A capability-based processor architecture demonstrating that LAMBDA CR6 is the optimal recursion primitive — superior to both conventional branch loops and CALL-based recursion. The CALL instruction's lump split naturally establishes a self-reference in the capability list register (CR6). LAMBDA CR6 exploits this self-reference through idempotent re-entry: LAMBDA CR6 is permitted to re-execute while lambda_active is already set because the return address is invariant — the same value overwrites the same register. The software's own recursion argument drives the countdown; the hardware's existing 1-bit flag and PC register handle exit: the base-case RETURN clears the flag and jumps to the instruction after LAMBDA CR6, where a second RETURN pops the CALL frame. Two RETURNS total, regardless of recursion depth. No hardware counter, no stack frames, no additional registers. CALL CR6 also re-enters the same method but with full namespace gate re-validation — which is redundant because the gate re-checks the same GT and re-splits the same lump every time, adding $O(N)$ cost with no security benefit. LAMBDA CR6 dominates: identical security (same method, same domain, same capabilities), $O(1)$ cost versus $O(N)$, zero additional hardware, and elimination of all branch-prediction vulnerabilities (Spectre, Meltdown, pipeline stalls). The architecture is extended with a natural language (English) front-end for the CLOOMC++ compiler. Pet-name mathematical constants (Pi, E, Phi) are accessed as Abstract Golden Tokens through the capability system.